Autodiagnóstico de Ingeniería del Diseño del Módulo 2

Motor de Asignación Automática de Horarios de Clases

Jesús David Beltrán Zárate

Juan Sebastián Albarracín Isaza

Daniel Felipe Delgadillo Sepúlveda

Martín Daniel Porto Gutiérrez

Programa de Ingeniería de Sistemas, Facultad de Ingeniería, Universidad El Bosque

Proyecto Núcleo 2 (15319), 2026-2, Grupo 5

Ing. Samuel Galindo Pinzón

23 de octubre de 2026

Introducción

Este documento recoge el autodiagnóstico que hicimos sobre el Documento de Diseño de Software que entregamos en el Módulo 2. Partimos de la pregunta orientadora de la actividad y revisamos qué tan seguros estamos de las decisiones que tomamos. Primero cada uno respondió su reflexión individual. Con esas respuestas elegimos tres decisiones de diseño y las contrastamos con la evidencia del Documento Maestro y del Documento de Anexos. También revisamos el código del motor que acompaña el entregable. Al final definimos tres ajustes que queremos tener listos antes del 5 de noviembre.

Reflexión Individual

Las cuatro reflexiones completas están en el Anexo A. Aquí resumimos lo que cada integrante identificó como fortaleza y lo que necesita reforzar.

Jesús David Beltrán Zárate se apoyó sobre todo en Ingeniería de Software 1 y 2 para entender los patrones de diseño y la arquitectura por capas. También usó Programación 1 y 2 y Bases de Datos 1 y 2 y aplicó Gestión de Proyectos al escoger Strategy y las tecnologías y al plantear los prototipos. La decisión que mejor defiende son las dos restricciones UNIQUE de la tabla de asignación porque hacen que la base de datos también bloquee los choques de docente y de aula. Su duda es el mecanismo por el que el orden ingenuo se trabó en una de las corridas. Necesita reforzar Angular y JWT.

Juan Sebastián Albarracín Isaza tomó como base Programación 1 y 2 para las pruebas y lo que ha visto en Ingeniería de Software 1 y 2 sobre las fases de desarrollo. Puede sustentar bien la arquitectura modular en cuatro capas con el frente en Angular y la API y el backend en Java y la base en PostgreSQL. Reconoce que no sabría defender las restricciones UNIQUE ni el diagrama de secuencia porque no participó en esas partes. Necesita repasar Strategy y los patrones en general y la conexión entre la API y el frente.

Martín Daniel Porto Gutiérrez se apoyó en ingeniería de requisitos y en arquitectura y patrones de diseño y en el modelado de la base de datos. Defiende con seguridad el alcance y los

roles de los stakeholders y la lógica del negocio y también por qué Strategy sirve en este proyecto. Su duda principal es la seguridad porque siente que le falta base para defender la autenticación y la autorización y el manejo de datos sensibles. Necesita repasar a fondo los diagramas para poder llevarlos a código.

Daniel Felipe Delgadillo Sepúlveda usó Programación Orientada a Objetos para expresar las restricciones como clases intercambiables y Estructuras de Datos para razonar el costo de una lista frente a un mapa. Bases de Datos le sirvió para el modelo normalizado y para las restricciones UNIQUE e Ingeniería de Software para justificar la arquitectura. Defiende Strategy con sus alternativas descartadas y la prueba de RD-09 y también la heurística de la sesión más restringida. Sus dudas son las más técnicas del equipo. Encontró que el umbral de RV-01 en el código no coincide con el del Módulo 1 y que varias afirmaciones del documento no tienen evidencia de ejecución. Necesita reforzar propagación de restricciones y pruebas con JUnit y pruebas de carga.

Al juntar las cuatro respuestas vemos dos cosas. La primera es que Strategy es la decisión que más integrantes pueden defender y eso nos da confianza en ella. La segunda es que el conocimiento del diseño está repartido de forma desigual. Cada uno domina la parte que trabajó y tiene huecos en las demás. Eso es un riesgo para la verificación individual y para la construcción del producto y lo tratamos en las Secciones 3 y 4.

Decisiones Analizadas

Elegimos las tres decisiones de las que más depende que el producto funcione. Para cada una revisamos el problema que resuelve y sus argumentos técnicos y la evidencia que la sustenta. También pensamos qué pasaría si estuviera mal y si la mantenemos.

Decisión 1. Restricciones Duras con el Patrón Strategy

El motor tiene que revisar cinco restricciones duras en cada tupla candidata y el sistema debe poder agregar reglas nuevas sin modificar el motor (RNF-03). Además cada rechazo tiene que decir qué regla falló (RNF-04). Esta decisión resuelve esas dos necesidades.

El argumento técnico es que cada restricción es un algoritmo distinto que el motor usa siempre de la misma forma. Por eso el motor depende de la interfaz RestriccionDura y no de las clases concretas y así cumple el principio abierto cerrado y el de inversión de dependencias. Además cada estrategia trae su código y el reporte de conflictos sale solo. En el documento comparamos contra no usar patrón y contra Chain of Responsibility y Template Method.

La evidencia está en las Secciones 6 y 7 del Documento Maestro y en la Figura A1 de los Anexos. La más fuerte es la prueba de cambio de la Tabla 3 del Maestro. Agregamos RD-09 como clase nueva y el motor generó las 40 sesiones sin que ningún grupo quedara con dos sesiones el mismo día. RD-09 rechazó 360 tuplas en el camino.

Si la decisión fuera incorrecta cada regla nueva obligaría a editar y volver a probar la clase más delicada del sistema. El reporte de conflictos tampoco podría decir la causa y el coordinador vería un error sin explicación.

La mantenemos tal como está. Lo que sí debemos completar es la evidencia. El documento dice que la comparación de versiones solo muestra la clase nueva y la línea de registro pero esa comparación no aparece en los Anexos. Al revisarla vimos que MotorAsignacion no cambió pero sí cambiaron ConfiguracionParametros y el programa de prueba por los otros dos cambios. Hay que mostrarla completa y corregir esa frase. También queda sin medir aparte el costo de la llamada polimórfica aunque el tiempo total en 3x nunca pasó de 53 milisegundos con la variante elegida.

Decisión 2. Ordenar las Sesiones por la Más Restringida Primero

El motor tiene que entregar un horario en un tiempo acotado aun con datos difíciles (RNF-01 y RNF-02). El problema es combinatorio y el orden en que se toman las sesiones decide cuánto tiene que retroceder el backtracking.

El argumento técnico viene de la literatura de satisfacción de restricciones. Atender primero la variable con menos opciones hace que los fallos aparezcan pronto y no muchas

sesiones después. En nuestro caso las sesiones con menos opciones son las que exigen aulas escasas como las salas con computadores.

La evidencia está en la Tabla 2 del Maestro y en el Anexo F. En promedio el orden ingenuo fue más rápido porque no paga el conteo inicial. Pero en el perfil exigente una instancia de 1x hizo que el orden ingenuo explorara 488.428.098 nodos y se detuviera a los 15 segundos. La misma instancia con la heurística terminó en 3 milisegundos y 25.656 nodos. La explicación es que el orden ingenuo fue ocupando las pocas salas con computadores con cursos que no las necesitaban. Cuando llegaron los cursos que sí las exigían ya no quedaban y el error estaba tantas sesiones atrás que el retroceso tuvo que probar millones de combinaciones.

Si la decisión fuera incorrecta un semestre con pocas aulas especiales podría dejar al coordinador esperando sin respuesta. Eso es justo lo que RNF-01 busca evitar.

La mantenemos pero hay que revisar su alcance. Solo probamos cinco semillas por perfil con datos sintéticos que generamos nosotros. El orden además es fijo y no se recalcula durante la búsqueda y tampoco hay verificación hacia adelante. Por eso no sabemos si alcanza con datos reales más grandes o más difíciles. También vimos que el conteo inicial se lleva entre el 50 y el 81 por ciento del tiempo total.

Decisión 3. Arquitectura en Capas con el Motor en Memoria y Persistencia Relacional

El sistema tiene que ser web porque así lo pide el documento base y tiene que cuidar dos atributos que se oponen. El motor debe ser rápido y las reglas deben poder cambiar sin tocar el resto. También hay que guardar quién generó cada horario (RNF-06).

El argumento técnico es que las capas dejan el motor aislado de la interfaz y de los archivos. El motor trabaja solo en memoria y no hace ningún acceso a disco ni a red dentro del ciclo de búsqueda. La persistencia usa un modelo en tercera forma normal y la tabla de asignación tiene dos restricciones UNIQUE. Una impide que un docente tenga dos sesiones en la misma franja y la otra impide lo mismo con un aula. Así la base de datos también protege RD-01 y RD-02.

La evidencia está en la Sección 3 y la Sección 10 del Maestro y en las Figuras D1 y E1 y la Tabla D1 de los Anexos.

Si la decisión estuviera mal tendríamos un motor lento o acoplado a la interfaz y cualquier cambio de formato terminaría tocando el algoritmo. Si la persistencia no queda como se describe la trazabilidad de RF-01 a RF-04 y de RF-09 se queda sin respaldo en el código.

El diseño lo mantenemos pero hay que revisar lo que afirmamos sobre él. Las capas de aplicación y persistencia solo existen en los diagramas. El código actual no tiene CargadorInventariosCSV ni ExportadorHorarioCSV ni ControladorHorarios y aun así la Sección 5.2 del Maestro dice que el cargador traduce día y hora al índice de franja. El formato JSON aparece como opción pero no está especificado. La seguridad del perfil administrador tampoco tiene un diseño detallado y es la principal duda de Martín.

Diagnóstico del Equipo

Fortalezas

La mayor fortaleza es que las decisiones centrales están respaldadas por mediciones que se pueden repetir. El motor corre con semillas fijas y los nodos y las comparaciones no dependen del equipo donde se ejecute. Strategy no quedó solo en el papel porque lo probamos con una regla nueva. La matriz de trazabilidad une cada restricción del Módulo 1 con su requisito y su componente sin dejar ninguna suelta. También vemos como fortaleza que el documento reconoce que las cifras del dossier de la Fase 2 no se pudieron reproducir y explica por qué cambió la lectura del riesgo.

Debilidades

La primera debilidad la encontró Daniel. El Módulo 1 define RV-01 con el techo de CAP_MAX por 1,10 pero el código usa el piso. Con el tope de 40 ambos dan 44 y por eso no se notaba. Con el tope de 35 el código da 38 y lo correcto es 39. Lo verificamos con el conjunto de prueba y en ese caso no cambia el resultado porque los tres grupos marcados tienen 40 y 41 y 43

estudiantes. Aun así el número que publicamos en la Sección 5.2 está mal y la regla no se comporta como la especificamos.

La segunda debilidad es que el documento afirma cosas de la implementación que todavía no tienen evidencia. Ya la explicamos en la Decisión 3 con el cargador y el exportador y el controlador y en la Decisión 1 con la comparación de versiones de RD-09. La tercera es que la interfaz solo existe como wireframes de fidelidad media. La Sección 8 se apoya en una prueba con pocos usuarios y no tiene el registro de esa prueba como anexo.

Incertidumbres

No sabemos si la heurística con orden fijo y sin verificación hacia adelante alcanza con datos reales del programa. Todas las mediciones se hicieron con datos sintéticos de nuestro propio generador y un horario real puede tener restricciones más cruzadas. Tampoco hemos medido el costo de la llamada polimórfica de Strategy por separado ni el tiempo del sistema completo con la API y la base de datos.

Riesgos

El riesgo más importante es el conocimiento desigual. Juan Sebastián no domina las restricciones UNIQUE ni el diagrama de secuencia. Jesús no sabe explicar todavía el mecanismo del orden ingenuo. Martín no se siente seguro con la seguridad del sistema. Eso afecta la verificación individual y también la construcción porque cada parte depende de las otras. El segundo riesgo es el tiempo. Angular y JWT y la conexión entre la API y el frente son temas nuevos para varios de nosotros y el Módulo 3 exige construir y probar todo. El tercero es que el historial del repositorio todavía no refleja el aporte de cada integrante y eso pesa en la auditoría de commits.

Plan de Mejora

Priorizamos los tres ajustes que más afectan la coherencia del diseño y la viabilidad del producto. Ninguno cambia el alcance aprobado. Los tres buscan que lo que el documento afirma

quede respaldado por el código y por pruebas. La fecha límite de los tres es el 5 de noviembre de 2026.

Ajuste 1. Umbral de RV-01

El hallazgo es que el código calcula el umbral de RV-01 con el piso y el Módulo 1 lo especifica con el techo. Por eso con el tope de 35 publicamos 38 cuando lo correcto es 39. La acción es cambiar el método calcularUmbralMaximoGrupo para que use el techo como dice el Módulo 1 y después repetir la prueba de cambio y corregir la Sección 5.2 y la Tabla 3 del Documento Maestro. El responsable es Daniel Felipe Delgadillo Sepúlveda. Sabremos que quedó bien cuando una prueba con JUnit confirme que el umbral es 44 con tope de 40 y 39 con tope de 35 y cuando la nueva salida de la prueba de cambio y el Maestro corregido muestren el valor de 39.

Ajuste 2. Evidencia de las Capas de Aplicación y Persistencia

El hallazgo es que las capas de aplicación y persistencia solo existen en los diagramas y que el documento afirma cosas de la implementación que todavía no tienen evidencia. La acción es implementar CargadorInventariosCSV y ExportadorHorarioCSV con los formatos de la Tabla D2 de los Anexos junto con ControladorHorarios. También vamos a agregar al Anexo F la comparación de versiones de RD-09 y vamos a especificar el formato JSON o validar con el docente que el MVP quede solo con CSV. El responsable es Juan Sebastián Albarracín Isaza. Sabremos que quedó bien cuando unas pruebas con JUnit carguen los archivos de ejemplo y exporten el horario sin errores y cuando la comparación de versiones del anexo muestre que MotorAsignacion no cambió.

Ajuste 3. Seguridad del Perfil Administrador

El hallazgo es que la seguridad del perfil administrador que exige RNF-06 no tiene un diseño detallado y que el equipo todavía no la domina. La acción es diseñar e implementar la autenticación con Spring Security y JWT y guardar las contraseñas con bcrypt en la tabla usuario. Después vamos a explicar el flujo al resto del equipo en una sesión interna. Los

responsables son Martín Daniel Porto Gutiérrez y Jesús David Beltrán Zárate. Sabremos que quedó bien cuando una prueba confirme que generar un horario sin token responde 401 y con un token de administrador responde 200 y cuando quede el registro de la sesión interna.

Para el riesgo del conocimiento desigual también acordamos una práctica fija. Antes del 5 de noviembre cada integrante explicará a los demás la parte que mejor domina. Daniel explicará la heurística y el caso del orden ingenuo. Jesús explicará las restricciones UNIQUE y el modelo de datos. Juan Sebastián explicará la arquitectura y Martín el alcance y los diagramas una vez los repase. Cada integrante hará además sus propios commits sobre su parte para que el historial del repositorio muestre el aporte real de todos.

Anexo A. Reflexiones Individuales Completas

Las respuestas se transcriben como las escribió cada integrante. Solo se corrigieron errores menores de escritura.

Jesús David Beltrán Zárate

¿Qué conocimientos adquiridos durante mi carrera fueron determinantes para diseñar la solución? Usé mucho lo que vi en Ingeniería de Software 1 y 2 sobre todo los patrones de diseño y los patrones arquitectónicos. Gracias a eso entendí por qué Strategy servía para las restricciones y por qué convenía separar el sistema por capas. Programación 1 y 2 me dieron la base para entender las clases y cómo se conectan entre sí. Bases de Datos 1 y 2 me ayudaron con el modelo entidad relación y la normalización. Gestión de Proyectos me sirvió para llevar el proyecto con orden y lo tuve en cuenta al escoger Strategy y las tecnologías y al plantear los prototipos.

¿Qué decisión técnica del equipo puedo explicar y defender con mayor seguridad? Lo que mejor sé explicar son las dos restricciones UNIQUE de la tabla de asignación. La primera evita que un docente quede con dos clases a la misma hora y la segunda evita que dos clases queden en la misma aula a la misma hora. Las dos incluyen la ejecución porque el sistema guarda varias corridas del horario. Me parece necesario porque así la base de datos también bloquea los choques y no depende solo del motor.

¿Qué aspecto del diseño me genera dudas o no sabría defender suficientemente ante una revisión técnica? No tengo claro por qué el orden ingenuo se trabó en uno de los casos de prueba mientras la heurística lo resolvió casi al instante. Es lo que más necesito entender porque es la razón principal por la que mantenemos la heurística. Fuera de eso siento que el diseño está bien justificado y que hicimos un trabajo sólido.

¿Qué conocimiento necesito recuperar o fortalecer para contribuir mejor a la construcción del producto? Me falta mejorar en Angular porque con eso vamos a hacer la interfaz. También necesito aprender JWT que seguramente usaremos para el inicio de sesión del

administrador. Lo voy a reforzar poco a poco preguntándole a la inteligencia artificial y practicando con ejemplos pequeños.

Reflexión final. Además de saber programar creo que lo más importante es aplicar bien los patrones y las arquitecturas junto con lo que aprendí en Gestión de Proyectos. Si el sistema no está bien organizado y el trabajo no se planea con orden el código solo no basta para llegar a una solución sólida.

Juan Sebastián Albarracín Isaza

¿Qué conocimientos adquiridos durante mi carrera fueron determinantes para diseñar la solución? Para poder realizar esta entrega nosotros tomamos en cuenta todos los conocimientos de Programación 1 y 2 para las pruebas. Como también Ingeniería de Software 1 y lo que hemos ido desarrollando en Software 2 a la hora del conocimiento de las fases y el cómo se desarrolla.

¿Qué decisión técnica del equipo puedo explicar y defender con mayor seguridad? Puedo sustentar de una buena manera la arquitectura que nosotros elegimos, la cual fue una arquitectura modular. Nosotros pensamos en hacer una arquitectura en 4 capas con front en Angular, API de conexión, backend en Java y base en PostgreSQL.

¿Qué aspecto del diseño me genera dudas o no sabría defender suficientemente ante una revisión técnica? Restricciones UNIQUE, porque esta parte no fue en la que participé con mayor enfoque. Como también no hice el diagrama de secuencia.

¿Qué conocimiento necesito recuperar o fortalecer para contribuir mejor a la construcción del producto? Necesito recordar un poco sobre el patrón de diseño Strategy y los patrones en general, como también sobre las conexiones de la API con el front.

Martín Daniel Porto Gutiérrez

¿Qué conocimientos adquiridos durante mi carrera fueron determinantes para diseñar la solución? Para el diseño de la solución me sirvió bastante lo que hemos visto de

ingeniería de requisitos (todo el tema de levantar los stakeholders y casos de uso), la parte de arquitectura y patrones de diseño, y también cómo estructurar y modelar bien la base de datos.

¿Qué decisión técnica del equipo puedo explicar y defender con mayor seguridad? Lo que tengo más claro y puedo defender sin problema es cómo definimos el alcance, los roles de los stakeholders y la lógica principal del negocio. Además, entiendo muy bien y puedo justificar técnicamente por qué decidimos usar el patrón de diseño Strategy y cómo nos soluciona la vida en este proyecto.

¿Qué aspecto del diseño me genera dudas o no sabría defender suficientemente ante una revisión técnica? Para ser sincero, donde más dudo es en la parte de seguridad. Si me hacen una revisión muy técnica sobre la autenticación, la autorización o cómo estamos manejando los datos sensibles, siento que todavía me falta base para defenderlo con seguridad.

¿Qué conocimiento necesito recuperar o fortalecer para contribuir mejor a la construcción del producto? Me di cuenta de que necesito sentarme a repasar a fondo los diagramas que ya diseñamos. Tengo que entender su estructura a la perfección para poder aportar mucho más ahora que nos toca traducir todos esos modelos a código real.

Daniel Felipe Delgadillo Sepúlveda

¿Qué conocimientos adquiridos durante mi carrera fueron determinantes para diseñar la solución? Programación Orientada a Objetos fue clave para expresar las restricciones como interfaces y clases intercambiables (patrón Strategy). Estructuras de Datos me permitió razonar el costo de recorrer una lista frente a consultar un mapa por franja y anticipar las cifras de las mediciones. Bases de Datos fue la base del modelo entidad relación normalizado y de proteger los choques con restricciones UNIQUE. Ingeniería de Software me dio el vocabulario de atributos de calidad para justificar la arquitectura en capas.

¿Qué decisión técnica del equipo puedo explicar y defender con mayor seguridad? La separación de las reglas con Strategy. Puedo explicar el problema (agregar o explicar reglas sin tocar el motor), las alternativas descartadas (Chain of Responsibility y Template Method) y la

evidencia, que es la prueba de cambio con la regla RD-09 y los códigos de regla en el reporte de conflictos. También defiendo mantener el ordenamiento por sesión más restringida porque en las mediciones acota el peor caso. El orden ingenuo agotó los 15 segundos en una instancia exigente.

¿Qué aspecto del diseño me genera dudas o no sabría defender suficientemente ante una revisión técnica? En RV-01 el código usa Math.floor y da 38, pero la especificación del Módulo 1 usa el techo y da 39 con tope 35. Está documentado pero sin resolver y no sé cuántos grupos marcaría el umbral correcto. También hay afirmaciones de implementación sin evidencia. Una es que el motor no cambió al agregar RD-09, porque no hay comparación de versiones en los Anexos. Otra es que la carga y exportación CSV funcionan, porque no hay salida de ejecución. Y otra es que JSON es compatible, porque no está especificado ni implementado. En el algoritmo el ordenamiento es estático y no hay verificación hacia adelante, así que no sé si seguiría siendo suficiente con datos más grandes o difíciles. En la interfaz son solo wireframes y las pruebas con usuarios son pocas.

¿Qué conocimiento necesito recuperar o fortalecer para contribuir mejor a la construcción del producto? Propagación de restricciones (verificación hacia adelante y consistencia de arcos) para reducir el retroceso en casos más difíciles. También pruebas automatizadas con JUnit y pruebas de carga, para respaldar con evidencia verificable lo que hoy afirmamos solo con mediciones manuales.